

- **Scene 1 - channel name**

[I plan to make this 2-3 quiet and start talking after this scene - when it comes to agenda]

- **Scene 2 - topic and agenda**

Welcome to The Art of Machine Learning. Today, we will talk about how a neural network learns one of the basic mathematical functions - linear functions.

This video will consist of five parts:

- **Short introduction to linear functions** - this will be a short reminder from school, about linear functions. I will show you the 2D function, 3D function, how parameters affect their graphs and I will introduce the generalized N-dimension formula in both dot product form and vectorized form
- **Creating, splitting and analysing the data** - here, we will jump into the code, where we will define an example linear function, create a dataset consisting of 100 samples (x, y), split this dataset into the *training and testing* data and finally, we will use **matplotlib** to plot both subsets
- **Building and training model** - in this step we will build a very simple model to learn the function we defined in the first step. I will also explain to you some of the most important concepts in deep-learning:
 - **Forward propagation**
 - **Gradient descent**
 - **Backpropagation**
 - **Loss function**
 - **Optimizer**
 - **Epochs**
 - **Learning rate**
- **Analysing model output** - here, we will compare models performance before and after training, we will check how performance improves depending on the amount iterations through training dataset. I will also plot the model predictions and we will see how off they are compared to the perfect fit
- **Saving the model** - often you want to save your model to use it later to train models with different structure, to share it with someone, whatever you want, so in this short part I will show you how to save your model into a file

In this video we will be using Python and PyTorch

One more thing before we dive into the main part - I know some of you might be more experienced with machine learning, but some people are not. That's why I will start with the very basics to build a foundation and we will introduce more and more advanced topics and techniques in further videos. I still strongly encourage you to watch it as a reminder. If you spot any mistakes or anything feels unclear to you, feel free to ask in comment section - I will definitely try to clarify anything for you and correct any mistakes as soon as possible

- **Scene 3 - Linear function - 2d case**

Let's start as planned, with a 2-dimensional linear function. As you can see, here is the general formula for it - $y = ax + b$, where **a** controls the steepness and the direction of the line, the larger absolute value of **a** is, the steeper the line climbs or drops. There is also **b** parameter which controls the vertical position of the line, it indicates the intersection of the graph with Y axis. Look at this graph and watch, how the change of parameters affects it.

- **Scene 4 - Linear function - 3d case**

Now, let's proceed to the 3-dimensional case of linear function. As you probably realized, in 2-d we were dealing with straight line, but now, we are dealing with plane. As previously, we have the general formula, which is quite similar to the 2-dimensional case, we just introduced the coordinate from one more axis with its corresponding weight. Take a look how a change of those parameters impact the graph of this function

- **Scene 5 - Linear function - generalised case for n-d**

You've probably already noticed the pattern which emerged by adding a 1 dimension, so why don't we try to generalise it for some **n** amount of dimensions? Here you can see the general formula for n-dimensions. We just take coordinates from x_1 to x_n axes, we multiply them by their corresponding weights and add bias. This formula seems pretty long, so we can try to convert it into way shorter **vector form**. Let's put weights into a vector. Do the same with x_1 to x_n coordinates. But here comes the problem. We can't just multiply those vectors (matrices), because their sizes are invalid for multiplication. Quick reminder - the inner size of left vector (matrix) must be equal to the outer size of right (vector) (matrix). So what about that? If we transpose the weights vector, we can conduct a multiplication $W^T * X$ and from here we are very close to our original **dot-product form**. What is missing? Of course it's bias, but we can just add it to the result of multiplication. That's how we end up with way cleaner and useful form. I strongly encourage you to perform this multiplication by hand and check that those forms are equal

Now, let's dive into a code

- **Google Colab - part 1 - Creating, splitting and analysing data**

I decided to use Google Colab, a free tool that comes with pre-installed machine learning libraries. More importantly, it gives us free access to cloud GPUs. While PyTorch does support AMD and Apple Silicon nowadays, NVIDIA's CUDA is still the absolute industry standard. Also it ensures, that you can follow along very easily.

Ok, let's finally start coding. We will code a model to learn typical 2d linear function. Let's define weight and bias. Now I'm creating dataset with torch.arange and to get y I'll use linear formula mentioned earlier. Now let's check the shape - everything is all right. Here, I'll split the data into:

- training dataset - used for training
- testing dataset - used for checking the performance of our model

The typical division is 80% for training, 20% for testing and that is what we're going to do in this case

****Note****: This time, we're going to make this by hand, but later we will be using the method build in `*sklearn.model_selection train_test_split*`

Let's check the size of each set - everything is all right. Let's also plot the data

• Google Colab - part 2.1 - Building a model

In this part, we will define the model architecture. Almost everything in PyTorch inherits from `nn.Module` so here we also have to do so. We define the constructor and inherit the constructor from `nn.Module`. How, we have the most basic way to define model parameters - `nn.Parameter`. `requires_grad` informs, that we want to adjust this parameter. We also have to override forward method, which basically says, how our model does the calculations. It starts with random values of parameters, which will be improved during training. Let's also check the values of random parameters before training, and try to make a predictions with this untrained model

****Note**: It is very important to set the manual seed to get reproducible results in case we wanted to improve our model or experiment with it

• Scene 6 - Training process

Before proceeding into writing the training loop, let's pause for a bit to go through the training visually. So here, on the left side we see the graph of the function with weight and bias randomly selected by the model. The yellow dots, are points we are trying to fit to. On the right side you can see the graph of the loss function in the case Mean Squared Error (the formula is below). So in the first step, we calculate the **loss (forward propagation)**. Then we calculate the gradient of the loss function which is vector of partial derivatives in respect of all the parameters (**back propagation**) - in this case, weight and bias. Then, we make a step in direction opposite to the gradient (**gradient descent**). The size of this step declared as **learning rate**. We repeat all of these steps for pre-defined amount of times called **epochs**. When it comes to learning rate, it is very important hyperparameter to choose. When it is too low - the model will learn very slow. When it is too high, the training will be unstable which may result in model not being able to find the loss function minima. It's very problem specific, and will come in practice. Let's now code all this logic in PyTorch.

- **Google Colab - part 2.2 - Training the model**

First, we have to choose the loss function - here we will use MSE and optimizer - which is an algorithm to perform the gradient descent. We put as an argument model parameters and `lr`, write in the number of epochs and write the right loop.

So we put model into training mode, make predictions on training data with actual state, calculate the loss, zero the gradients, perform backpropagation and optimizer step. Here, still inside the outer loop, we can perform testing, so let's put the model into evaluation mode, and using `torch.inference_mode`, let's make predictions on `test_set`. Now calculate loss on the test set. At the end, let's print the parameters, epoch, loss, and test loss, let's say every 10 epochs, so we will see what's actually happening.

(Optional) Also, let's define 3 optional arrays which will help us track loss. We will use them for plotting.

- **Google Colab - part 3 - Plotting result**

Now let's make prediction and plot the results on test dataset, but looking at model `state_dict` - it has definitely learnt something and it is pretty close. The moment of truth - nice. It's not perfect but it's way better than it was with random values. To improve it, when we have perfectly generated dataset, we can just improve the number of epochs. See? Now it's almost perfectly fit.

Let's also plot the loss function. Look at the graph - both train and test losses are going to zero, so I believe we did everything all right.

- **Google Colab - part 4 - Saving model to a file**

As the last thing, we will save our model to a file. To do that, we simply save model's state dict to a file. It is recommended to save PyTorch models in files with `.pt` or `.pth` extensions. We import `Path` from `pathlib`, define and create model catalogue, write model name, and save it using `torch.save`

- **Scene 7 - Ending**

Well, that was quite a journey. First, we reminded some math in regards of linearity. Then we defined the linear function, created some data and analysed it. We also build and trained the model, as well as analysed it's output and ended by saving it's to the file. I deeply encourage you to experiment with parameters and hyperparameters, for example change the learning rate, number of epochs, maybe Google what optimizers PyTorch offers and try different ones and same with loss functions. That will be enough for today. Regarding the fact that it is the first video on

this channel, any feedback will be much appreciated, so I will know how to improve in the future. Thank you for your attention.